

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Poster Presentation

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO3 –Poster Presentation	Assessment:	Assessment Tool 1– Poster Presentation
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	MYTHREA SHREE	Reg. No.:	192424089
Section / Batch:	2024	Date:	27/08/2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

		reasoning and insights				
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

```
colab.research.google.com/drive/1P5GkwOEjNGJOJs3K1dqwsgcQ-2exiRe3#scrollTo=bKA7r

Untitled5.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[4] ✓ 1s ▶
# -----
# 2. GINI IMPURITY
# -----

def gini_impurity(y):
    """
    Calculate Gini impurity.
    Gini = 1 - sum(p^2)
    """

    if len(y) == 0:
        return 0

    classes, counts = np.unique(y, return_counts=True)

    probabilities = counts / len(y)

    gini = 1 - np.sum(probabilities ** 2)

    return gini

# -----
# 3. SPLIT DATA
# -----

def split_data(X, y, feature, value, feature_type):
    """
```

colab.research.google.com/drive/1P5GkwOEjNGJOjs3K1dqwsgcQ-2exiRe3#scrollTo=bKA7roYK5khR

Untitled5.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

95% Confidence Interval = (9.825230/320/0919 , 14.9/4/6926/929082)

```
[4] ✓ 1s # =====
# CART CLASSIFICATION FROM SCRATCH
# Handles Numerical + Categorical Features
# Uses Gini Impurity and Recursive Binary Splitting
# =====

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# -----
# 1. SAMPLE DATASET
# -----

data = pd.DataFrame({
    'Age': [22, 25, 28, 35, 40, 45, 50, 23, 30, 38, 48, 55],
    'Income': [25, 30, 35, 50, 60, 70, 80, 28, 45, 55, 75, 90],
    'City': ['A', 'A', 'B', 'B', 'C', 'C', 'C', 'A', 'B', 'B', 'C', 'C'],
    'Buys': ['No', 'No', 'No', 'Yes', 'Yes', 'Yes',
            'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes']
})

print("Dataset:")
print(data)
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

[4]
✓ 1s

return x_left, y_left, x_right, y_right

# -----
4. FIND BEST SPLIT

def find_best_split(x, y, feature_types):

"""

Find the feature and value producing
the lowest weighted Gini impurity.

"""

best_gini = float("inf")

best_feature = None

best_value = None

for feature (variable) right_mask: Any

feature_type = feature_types[feature]

Numerical feature

if feature_type == "numerical":

values = sorted(x[feature].unique())

Commands + Code + Text Run all

[4] ✓ 1s

```
# Try midpoints between consecutive values
thresholds = [
    (values[i] + values[i + 1]) / 2
    for i in range(len(values) - 1)
]

# -----
# Categorical feature
# -----

else:
    (variable) values: list

    thresholds = X[feature].unique()

# -----
# Evaluate every split
# -----

for value in thresholds:

    X_left, y_left, X_right, y_right = split_data(
        X, y, feature, value, feature_type
    )

    # Ignore empty splits
    if len(y_left) == 0 or len(y_right) == 0:
        continue

    left_gini = gini_impurity(y_left)
```

Commands + Code + Text Run all

[4] ✓ 1s

```
left_gini = gini_impurity(y_left)
right_gini = gini_impurity(y_right)

# Weighted Gini
weighted_gini = (
    (len(y_left) / len(y)) * left_gini
    + (len(y_right) / len(y)) * right_gini
)

if weighted_gini < best_gini:

    best_gini = weighted_gini
    best_feature = feature
    best_value = value

return best_feature, best_value, best_gini

# -----
# 5. TREE NODE
# -----

class Node:

    def __init__(
        self,
        feature=None,
        value=None,
        ...
    ):
```

Variables Terminal

File Rename notebook Interrupt Runtime Tools Help

Commands + Code + Text ▶ Run all ▼

[4]
✓ 1s

▶

```
        value=None,
        feature_type=None,
        left=None,
        right=None,
        prediction=None
    ):

        self.feature = feature
        self.value = value
        self.feature_type = feature_type

        self.left = left
        self.right = right

        self.prediction = prediction

# -----
# 6. CART CLASSIFIER
# -----

class CART:

    def __init__(self, max_depth=5, min_samples_split=2):

        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.root = None
        self.feature_types = None
```

Variables Terminal



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[4]

✓ 1s



```
# -----  
# Determine feature types  
# -----  
  
def determine_feature_types(self, X):  
  
    self.feature_types = {}  
  
    for column in X.columns:  
  
        if pd.api.types.is_numeric_dtype(X[column]):  
  
            self.feature_types[column] = "numerical"  
  
        else:  
  
            self.feature_types[column] = "categorical"  
  
# -----  
# Majority class  
# -----  
  
def majority_class(self, y):  
  
    return y.value_counts().idxmax()  
  
# -----  
# Recursive tree building  
# -----
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[4]

✓ 1s



```
def build_tree(self, X, y, depth=0):

    # Stop if all samples belong to one class
    if len(np.unique(y)) == 1:

        return Node(prediction=y.iloc[0])

    # Stop conditions
    if (
        depth >= self.max_depth
        or len(y) < self.min_samples_split
        or len(X) == 0
    ):

        return Node(prediction=self.majority_class(y))

    # Find best split
    feature, value, gini = find_best_split(
        X, y, self.feature_types
    )

    # If no valid split exists
    if feature is None:

        return Node(prediction=self.majority_class(y))

    # Split data
    x_left, y_left, x_right, y_right = split_data(
        X,
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

[4]
✓ 1s

```
        X,
        y,
        feature,
        value,
        self.feature_types[feature]
    )

    # Recursively create left subtree
    left_subtree = self.build_tree(
        X_left,
        y_left,
        depth + 1
    )

    # Recursively create right subtree
    right_subtree = self.build_tree(
        X_right,
        y_right,
        depth + 1
    )

    # Return decision node
    return Node(
        feature=feature,
        value=value,
        feature_type=self.feature_types[feature],
        left=left_subtree,
        right=right_subtree
    )
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[4]

✓ 1s



```
# -----  
# Train model  
# -----  
  
def fit(self, X, y):  
  
    self.determine_feature_types(X)  
  
    self.root = self.build_tree(X, y)  
  
# -----  
# Predict one sample  
# -----  
  
def predict_sample(self, sample, node):  
  
    # Leaf node  
    if node.prediction is not None:  
  
        return node.prediction  
  
    # Numerical feature  
    if node.feature_type == "numerical":  
  
        if sample[node.feature] <= node.value:  
  
            return self.predict_sample(  
                sample,  
                node.left
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼



[4]

✓ 1s



```
    )

    # Categorical feature
    else:

        if sample[node.feature] == node.value:

            return self.predict_sample(
                sample,
                node.left
            )

        else:

            return self.predict_sample(
                sample,
                node.right
            )

# -----
# Predict multiple samples
# -----

def predict(self, X):

    predictions = []

    for _, sample in X.iterrows():
```

```
        prediction = self.predict_sample(
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[4]

✓ 1s



```
        prediction = self.predict_sample(
            sample,
            self.root
        )

        predictions.append(prediction)

    return np.array(predictions)

# -----
# 7. PREPARE DATA
# -----

X = data.drop('Buys', axis=1)
y = data['Buys']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# -----
# 8. TRAIN CART
# -----
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[4]

✓ 1s



```
# -----  
# 8. TRAIN CART  
# -----  
  
model = CART(  
    max_depth=4,  
    min_samples_split=2  
)  
  
model.fit(X_train, y_train)  
  
# -----  
# 9. PREDICTION  
# -----  
  
y_pred = model.predict(X_test)  
  
print("\nActual values:")  
print(y_test.values)  
  
print("\nPredicted values:")  
print(y_pred)  
  
# -----  
# 10. ACCURACY  
# -----  
  
accuracy = accuracy_score(y_test, y_pred)
```



Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands

+ Code ▾

+ Text

▶ Run all ▾

[4]
✓ 1s

```
# -----  
# 11. DISPLAY TREE  
# -----  
  
def print_tree(node, depth=0):  
  
    spacing = "  " * depth  
  
    # Leaf  
    if node.prediction is not None:  
  
        print(  
            spacing +  
            "Predict -> " +  
            str(node.prediction)  
        )  
  
        return  
  
    # Decision node  
    if node.feature_type == "numerical":  
  
        condition = (  
            f"{node.feature} <= {node.value}"  
        )  
  
    else:  
  
        condition = (  
            f"{node.feature} == '{node.value}'"
```




Untitled5.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text ▶ Run all ▾



[4]

✓ 1s



```
print(spacing + "IF " + condition)

print(spacing + " THEN:")

print_tree(node.left, depth + 1)

print(spacing + " ELSE:")

print_tree(node.right, depth + 1)

print("\n===== CART TREE =====")

print_tree(model.root)
```



... Dataset:

	Age	Income	City	Buys
0	22	25	A	No
1	25	30	A	No
2	28	35	B	No
3	35	50	B	Yes
4	40	60	C	Yes
5	45	70	C	Yes
6	50	80	C	Yes
7	23	28	A	No
8	30	45	B	Yes
9	38	55	B	Yes
10	48	75	C	Yes
11	55	90	C	Yes

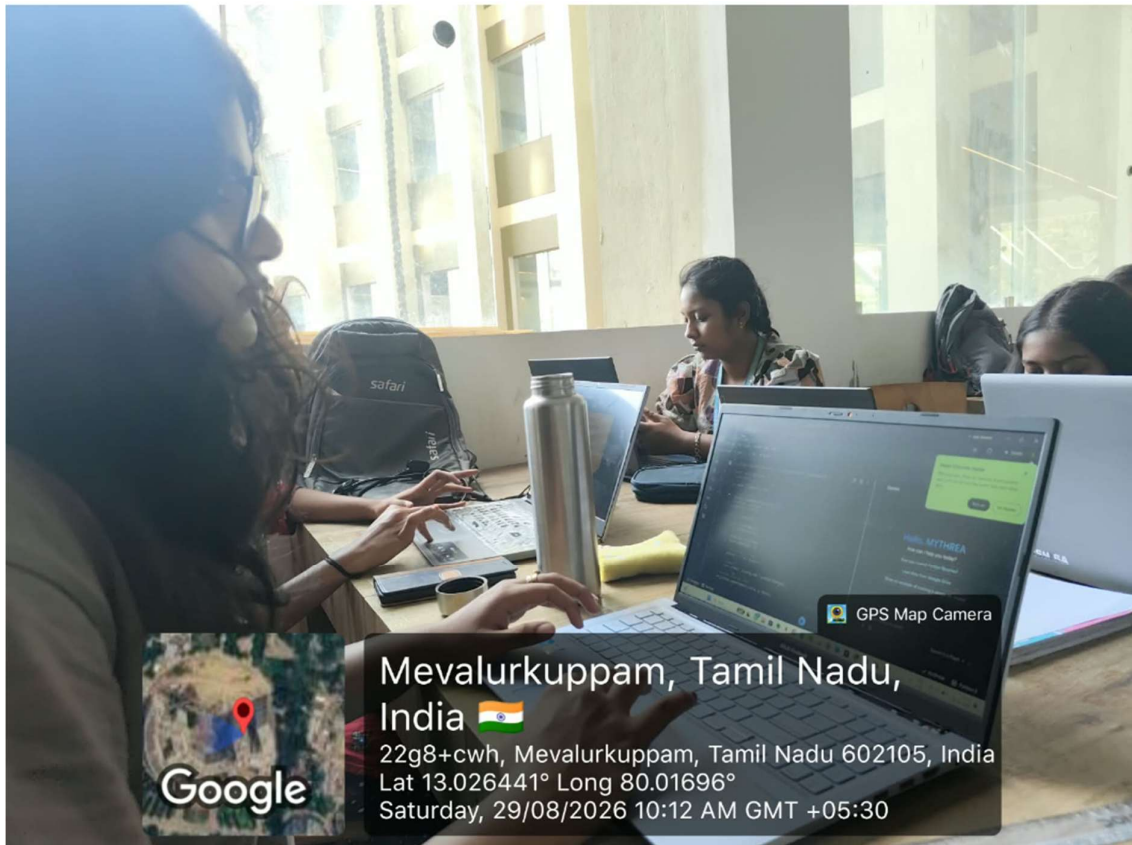


Actual values:
['Yes' 'Yes' 'No']

Predicted values:
['Yes' 'Yes' 'No']

Accuracy: 1.0

```
===== CART TREE =====
IF Age <= 29.0
THEN:
  Predict -> No
ELSE:
  Predict -> Yes
```



Mevalurkuppam, Tamil Nadu,
India 🇮🇳

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026441° Long 80.01696°
Saturday, 29/08/2026 10:12 AM GMT +05:30

Google